A Graph-based Framework for Coverage Analysis in Autonomous Driving

Thomas Mühlenstädt Marius Bause

Motivation and Existing Approaches

Motivation

- Coverage analysis is crucial to ensure safety and reliability of autonomous driving systems
- Coverage argument: have the observed scenes exhausted the space of traffic scenes that can occur?
- Existing arguments are typically collected **per coverage factor**, or up to 2–3 factor interactions
- This work: take the **complete traffic scene** as the unit of coverage
  - each scene → one graph
  - nodes carry actor parameters, edges carry relation parameters
  - arbitrarily complex scenes (varying numbers of actors, higher-order interactions) captured in one object, not decomposed into isolated factors
- Scope: completeness of observed scenes, *not* safety of the system

Existing Approaches

- Scenario-based testing is the dominant paradigm (PEGASUS: functional / logical / concrete scenario levels)
- **Approval trap**: statistically demonstrating superior safety needs billions of test km
- → motivates simulation and systematic coverage methods
- Standardized coverage criteria: ISO 21448, UL 4600
- Domain-specific coverage buckets (Foretellix)
- Recent trend: represent traffic scenes as **graphs** for scalable, label-free analysis
  - semantic scene-graph embeddings, contrastive learning, heterogeneous scenario graphs
- Gap: these graph representations are not designed specifically for coverage analysis → this work

Datasets

- **Argoverse 2.0**: large-scale real-world dataset
  - 250,000 scenarios, 11 s each
  - tracked trajectories (vehicles, pedestrians, cyclists) + semantic map
  - subset of 17,000 training scenes used
- **CARLA** 0.9.15: open-source simulator
  - six maps, 20–60 vehicles per run, mixed traffic
  - 2,050 scenes of 11 s each

Traffic Scene Graph and Construction Algorithm

Defining a Traffic Scene Graph

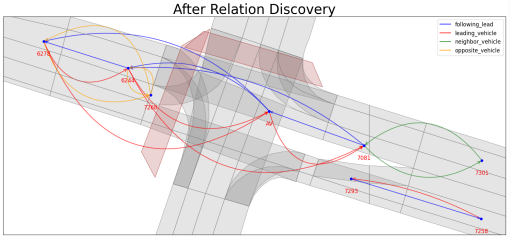
- **Map graph** G_{map}: nodes = lane segments (geometry + semantics)
- Map graph edges: **following**, **neighbor**, **opposite**
- **Actor graph** G_{actor}: nodes = actors at one timestep (lane, position, speed, type, lane-change indicator)
- Actor graph edges: relation type + path length
- Relation types (semantic hierarchy): **Following/Leading**, **Neighbor**, **Opposite**
- Lane-change indicator ($\Delta t = 1\,\text{s}$) captures cut-in/cut-out within a single-timestep graph

Hierarchical Graph Construction Algorithm

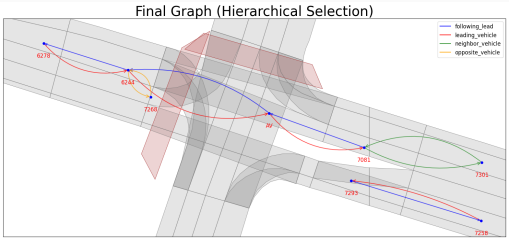
- **Phase 1 – Relation Discovery**: find a connecting path per actor pair; path structure sets relation type; filtered by distance/path-length thresholds
- **Phase 2 – Hierarchical Construction**: add edges leading/following → neighbor → opposite (shortest path first); skip an edge if a path already exists (BFS)

Relation	Dist. fwd/bwd [m]	Max. path [edges]
Leading/following	100	3
Neighbor	50 / 50	2
Opposite	100 / 10	2

Hierarchical Construction: Example



After relation discovery



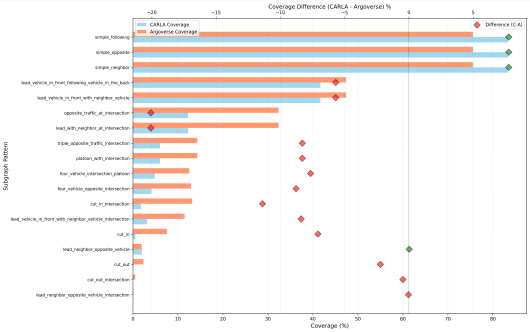
After hierarchical selection

The discovery graph (left) contains all relations within distance limits. Hierarchical selection (right) removes redundant edges representable through existing paths.

Coverage Method 1: Subgraph Isomorphism

- Traffic scene archetypes (lead-follow, cut-in, cut-out, opposite traffic, platoons, ...) expressed as small graphs
- Check whether a subgraph of scene graph G is isomorphic to archetype A (VF2 algorithm)
- Strategy:
  - define archetype set S
  - fix node/edge attributes considered
  - match each scene graph against $S \rightarrow$ coverage table C
- **18 archetypes** defined: 2-actor (3), 3-actor (9), 4-actor (4), 5-actor (2)
- Archetypes are user-defined, illustrative—not a complete or safety-certified taxonomy
- Real safety case: archetypes derived top-down (HARA / UL 4600), completeness of the set is critical

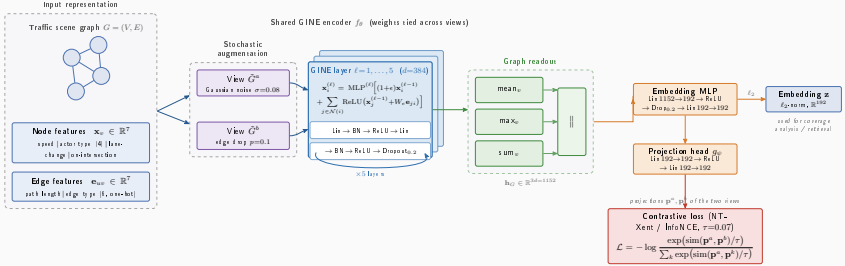
Results: Scenario Archetype Coverage Gaps



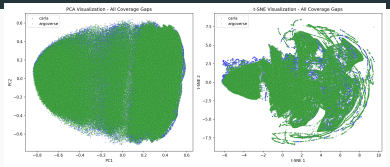
- CARLA higher coverage only for the 3 simple 2-actor scenarios
- All multi-actor scenarios: Argoverse shows higher occurrence
- `cut_out_intersection`: <1% CARLA vs. ∼8% Argoverse
- `lead_neighbor_opposite_vehicle_intersection`: ∼6% Argoverse, virtually absent in CARLA
- `cut_in`/`cut_out`: underrepresented in CARLA regardless of intersections

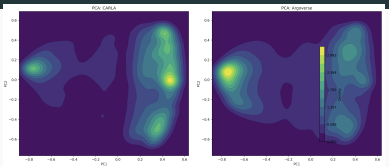
Coverage Method 2: Graph Embeddings (GINE)

- Real traffic scenes are far more complex than predefined archetypes → complementary top-down approach
- Graph Isomorphism Network with Edge features (GINE): natively encodes edge attributes (relation type + distance)
- Self-supervised contrastive training (NT-Xent/InfoNCE, $\tau = 0.07$) on CARLA + Argoverse jointly



Results: Embedding Space Analysis





- Plausibility check: nearest neighbours in embedding space share actor configuration and road geometry
- First two principal components explain 40% of the total variance
- CARLA peaks at higher PC1 values
- Argoverse peaks at negative PC1 values
- $\rightarrow$ datasets occupy different regions of the embedding space
- **23,148** Argoverse graphs identified as coverage gaps, dominated by
  - `cut_in` ($\sim$94% of all Argoverse cut-in instances)
  - `cut_out` and `cut_out_intersection`
  - `lead_neighbor_opposite_vehicle_intersection`

Summary and Future Work

- Graph-based framework: hierarchical scene graphs combining map topology with actor relationships
- Two complementary coverage methods:
  - subgraph isomorphism against archetypes $\rightarrow$ interpretable, pattern-based coverage
  - GINE embeddings via contrastive learning $\rightarrow$ similarity-based coverage, clustering, anomaly detection
- Validated on Argoverse 2.0 and CARLA: reveals structural, parametric, and co-occurrence coverage gaps in simulation
- Scales across scenarios without scenario-specific handling
- Naturally accommodates varying numbers of actors
- **Future work**:
  - multi-timestep temporal graphs
  - automatically extracting new archetype candidates from dense, unmatched embedding regions
- Code: `https://github.com/tmuehlen80/graph_coverage`